Distilling a Language-Model Decision Policy into Browser-Scale Students

How a small model of structured triples can stand in for a three-billion-parameter decision model in a town-building game

Shishir Tandale, with Claude

ABSTRACT

Hearthvale is a town-building game in which every resident's choices, from the daily routine to emigration and political protest, are made by a hosted decision model, LiquidAI's d1-3B. The approach gives towns varied and believable behaviour, but its cost grows with every player and with game speed, and a single shared GPU soon becomes the limiting resource. Because the game's questions are highly structured, we ask whether a small model running in each player's browser can reproduce the large model's answers. Building on the Triple World Model, an unpublished research system that represents state as sets of entity, attribute and value triples, we represent each question as an unordered set of triples and distil the teacher into several student architectures. A compact set model that attends over these triples matches the teacher most closely, generalizes to combinations of facts withheld from training far better than a conventional multilayer perceptron, and answers in roughly a tenth of a millisecond. Trained on states drawn from its own play as well as from broad simulation, in the manner of DAgger, it reproduces the teacher's answer distributions to within a few points on questions from towns the teacher governed, and towns under its control follow the same average course as the teacher's, although they vary somewhat less from one run to the next. A learned estimate of the student's own error then supports a continual training loop that learns from single players' questions within per-player limits, and a small budget of questions referred back to the teacher.

1 Introduction

1.1 A town that decides for itself

Simulation games have long relied on hand-written rules to animate their populations. Hearthvale takes a different course. The player governs a town as its council, reading a docket of proposals and ruling on them, while the

town itself makes thousands of smaller decisions: whether a resident works or rests this morning, whether a household stays, moves or sends for relatives, whether a faction backs a new tax. Each of these is put to a hosted decision model, LiquidAI's d1-3B, which reads a short description of the town and a question about a group of residents and returns a probability for every permitted answer. The game samples from that distribution, so a district of labourers might divide between work and rest in proportions that shift with the season, the food supply and the council's latest ruling. The result is behaviour that feels considered rather than scripted, and it is central to the game's character.

1.2 The cost of a hosted policy

This realism has a price that grows with success. Every question occupies time on a shared GPU, and the volume of questions rises with the size of each town, with the speed at which players run the game, and with the number of players. A city asks about eighteen questions a second at normal speed and several hundred at the highest speeds. Three costs follow. GPU time limits how many players one server can support. Network delay means answers arrive late, after the game has already moved on. And when the server falls behind, the game reverts to simple fallback rules, which are not a neutral substitute: towns governed by them grow several times larger than towns governed by the model, so a busy server visibly changes how the game plays.

Removing the decision model would remove these costs, and with them the quality that justifies it. The aim is instead to keep its behaviour while moving most of its work elsewhere. A student model running in each player's browser would answer most questions without the server, without delay and without a network connection at all. Because the student's purpose is to stand in for the teacher, it must copy the teacher's behaviour rather than invent its own, and so it learns only from the teacher's answers, never from outcomes in play.

1.3 Aim and contributions

The questions themselves suggest that this is feasible. Almost every input is categorical, such as whether food is low, whether rent is high or which institutions are active, and the same twenty-two kinds of question recur millions of times. A function with such a regular domain should be learnable by a model small enough to run in a browser. This report asks whether such a student can reproduce the teacher closely enough that a town under its control behaves like a town under the teacher, and at a cost an ordinary device can bear.

We make four contributions. First, we adapt the Triple World Model's representation of state as sets of triples to decision questions, and argue from its properties why this representation suits the problem. Second, we compare five families of student model on accuracy, cost and compositional generalization. Third, we propose an evaluation that goes beyond average error, measuring the student on the states its own policy reaches and comparing complete towns with the teacher's in both their average course and their spread. Fourth, we describe how the student continues to learn from the teacher after release without ever changing beneath a game in progress.

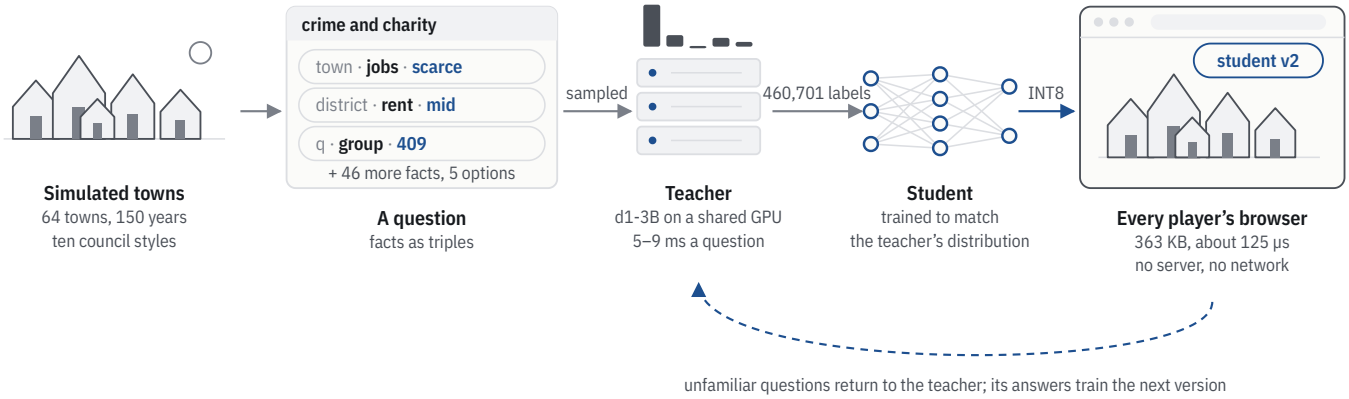


Figure 1. Overview of the approach. Questions gathered from simulated towns are labelled by the teacher, students are trained on the labels and shipped to the browser, and questions the deployed student finds unfamiliar return to the teacher for the next version.

2 Background

2.1 Distillation

The idea of training a small model to imitate a large one is well established. Hinton and colleagues showed that a compact network trained on the full output distribution of a larger network, rather than on hard labels alone, retains much of its accuracy, because the distribution conveys how the teacher weighs the alternatives [1]. Rusu and colleagues carried the same idea to agents that act, distilling reinforcement-learning policies into smaller networks [2]. Our setting is a direct descendant of policy distillation, with one difference that shapes everything else: the teacher here is a language model, and its inputs are structured records rather than images or game screens.

2.2 Learning on structured records

Structured records place the problem in the territory of tabular learning, where gradient-boosted decision trees such as XGBoost remain difficult to beat [3]. Neural approaches to tables commonly treat each column as a token and let a transformer relate them, as in FT-Transformer [4]. A known weakness of plain multilayer perceptrons is that they learn multiplicative interactions between features only with difficulty [5], which is why recommendation systems add explicit interaction terms. Factorization machines express every pairwise interaction as a product of learned feature vectors, and can therefore estimate the effect of a pair never observed together [6]. Set models, finally, treat their input as an unordered collection and are invariant to its order by construction [7].

2.3 Distribution shift and deferral

Two further strands bear on deployment. Imitation learning has long recognized that a copied policy visits states its teacher never showed it, and that its errors compound there; the DAgger algorithm addresses this by

collecting states from the student's own behaviour and labelling them with the teacher [8]. Selective prediction, in turn, allows a model to defer uncertain cases to a stronger one [9]. Both ideas reappear in the deployed system, the first in how training data is gathered and the second in how the student consults its teacher.

2.4 The Triple World Model

The work reported here grew out of an earlier research system, the Triple World Model, which has not been published and is therefore described in some detail. Its premise is that a small model can reason well about a world if the world is presented to it as structure rather than as prose. A state is written as an unordered set of triples, each naming an entity, one of its attributes and the attribute's value, such as a pet, its hunger and the level of that hunger. A compact transformer reads all the triples at once and predicts the next state, producing every output triple simultaneously rather than one after another, and learning the change from the current state rather than the state afresh.

The system was built to test one claim: that this decomposition allows generalization to combinations of facts never seen together in training. On a benchmark spanning three domains, it predicted the outcome for novel combinations of entity and state with an F1 score of 0.74, and ablations showed that attention across positions was necessary to do so. Where the correct output depended on other triples present in the state, it reached an F1 of 0.978, and a model of 178,000 parameters matched one of 4.5 million. The same architecture, reduced to 29,000 parameters, ran a pet simulation entirely in the browser in 303 kilobytes of JavaScript with 98.9% exact agreement. The underlying argument is that structural priors, namely decomposition into triples, invariance to order and a shared embedding space, carry knowledge that an unstructured model would otherwise have to acquire through parameters and data.

2.5 Decision models

A recent class of models answers typed questions about a document with a probability distribution over the permitted options, in one forward pass and without generating text, behind a common interface its vendors call System One. Liquid AI's d1, the teacher in this work, has 3.12 billion parameters. TypeSafe's Jev is closed. Cloudflare's Clef family includes Clef-flash, a 9-billion-parameter model derived from Qwen3.5-9B [10]. Kev, from Jared Palmer, comes in sizes from 0.8 to 27 billion parameters and adds a low-rank adapter and a pointer head that scores each option against the question to a Qwen base model [11]. Laya, from Convai Innovations, has 421 million parameters and is built on ModernBERT-large [12]. These models are general: they read free text and answer any question put to them. The student described here makes the opposite trade. At about 125,000 parameters it is three to five orders of magnitude smaller than any of them and answers only Hearthvale's twenty-two kinds of question, from structured input, but it needs no server. The difference matters at the scale of a game: at the hosted prices of October 2026 a million questions cost about $6 from d1 and $11 to $13 from Clef-flash, Jev or d1-ultra, and a single city at normal speed asks some 65,000 questions an hour.

3 Problem Setting

Each decision question consists of a kind, a shared snapshot and a short context. The kind names the decision, for instance a resident's routine, a household's choice to stay or leave, or a faction's stance on a proposal. The snapshot summarises the town, and for some kinds a district or a proposal, in coarse bands that change from one season to the next; many questions share the same snapshot. The context identifies who is asking: their trade, wealth, stage of life and immediate needs. The teacher's answer is a probability distribution over a fixed set of options for that kind, of which a question may permit a subset. Because the game samples from this

distribution, a student must reproduce all of it rather than merely its most likely option, and we therefore measure agreement by the Kullback–Leibler divergence from teacher to student, expressed in nats.

A satisfactory student must meet three conditions. It must answer close to the teacher, including in towns it has never seen, since players will build towns no training run anticipated. Towns under its control must behave like towns under the teacher, which is a stronger requirement than low average error, because the game samples from the student at every step for a century of game time and small biases accumulate. And it must be small and fast enough to run within the game's existing budget on an ordinary device, including the bursts of questions that open each game month.

4 Approach

4.1 Why the triple route

Hearthvale's decision questions are already structured records: a handful of categorical facts about a town, a district and the residents in question. The Triple World Model was designed for exactly this kind of input, and its properties correspond closely to the requirements set out above. Table 1 summarises the correspondence; the paragraphs below explain the most consequential points.

Table 1. Properties of the triple representation and the requirement each one serves.

Property	What it means here	Requirement it serves
Shared vocabulary	Every kind of question is written in the same terms, so one model answers all twenty-two	Size; maintenance
Order-free sets	Facts have no natural order, and lists of rules and institutions vary in length	Accuracy on varied towns
Multiplicative interactions	Attention relates pairs of facts, such as a faction and the interests a proposal serves	Accuracy
Compositional generalization	Answers remain sound for combinations of circumstances absent from training	Unseen towns
Small parameter count	The architecture has already run in a browser at tens of thousands of parameters	Device budget
Extension by tokens	A new fact is a new token and a new kind needs no feature code; the vocabulary can grow without retraining from scratch	Learning after release
A cacheable state	The snapshot, shared by many questions, can be encoded once and reused	Device budget; bursts

Two of these properties weighed most heavily. The first is compositional generalization. Players control the most consequential part of the input, the rules in force, and will combine them in ways no simulated council ever did; a student that merely memorises combinations seen in training would fail precisely where the game depends on it. The Triple World Model's central result was generalization of this kind, and it depended on attention between facts rather than on a fixed function of their sum. The second is economy. A single representation for every kind of question, derived from the schemas the game already uses, means one small model rather than twenty-two, one runtime in the browser and no feature engineering when a kind is added or changed.

The decision problem also differs from the world-modelling problem in ways that required adaptation. A world model predicts a whole next state, whereas a decision requires only a distribution over a few options, so the output becomes one score per option rather than a set of predicted triples, and there is no dynamics step. And whereas a world model's state may hold several entities that share attributes, each scope in a decision question describes a single entity, which allows a triple to be represented as one

token rather than three, as Section 4.3 explains. We kept the representation and the principle of order-free sets, and changed the output and the tokenization to fit the task.

4.2 Representation

Each question is rewritten as an unordered set of triples of the form scope, attribute and value, such as `(town, food, low)` or `(district, rent, high)`. A single function derives these triples from the schema that already defines each kind of question. Categorical fields become one triple each; integer fields with a small range are treated as categories; other numeric fields carry their value, compressed as sign(x)·ln(1+|x|), alongside the triple; and lists such as the active rules contribute one triple per item, together with a level where the item has one. The names of towns, districts and proposals are discarded, since nothing learned from a name transfers to another town. A value absent from the vocabulary maps to a shared unknown token, which also serves later as a signal that the student is outside its experience. Because the representation follows from the schema, a new kind of question requires no new feature code.

4.3 The set model and its alternatives

The principal student is a set model. Each triple is embedded as a single vector, the sum of embeddings for its scope, attribute and value, with numeric values added along a learned direction for their attribute. A small attention encoder without positional information reads the snapshot's triples; because its output depends only on the snapshot, the browser computes it once per snapshot and reuses it for every question that shares it. A second stage reads the question's own triples together with one token for each permitted option, each built from an embedding specific to that kind's option and one shared across kinds, so that an option such as leaving town can draw on what is known about leaving in other decisions. This

stage attends among its tokens and to the encoded snapshot, and each option token ends in one score; a softmax over the permitted options gives the distribution.

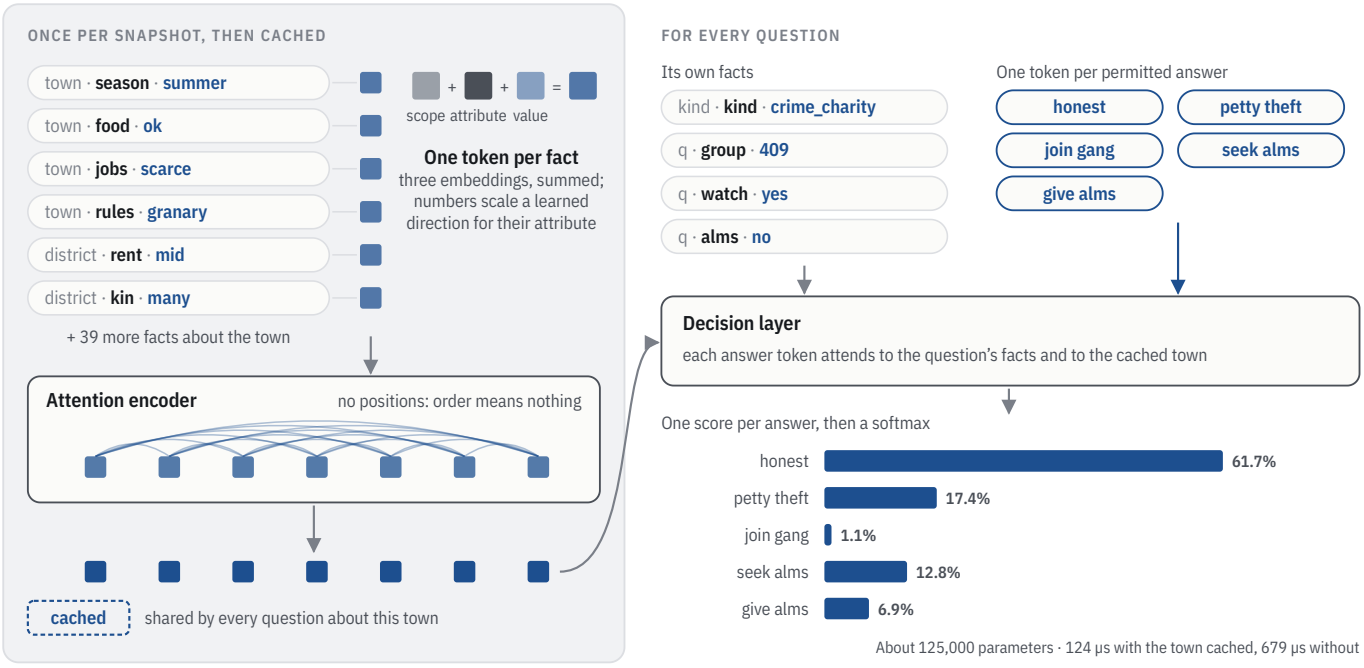


Figure 2. The set model, on one question: whether a group of 409 residents stays honest, turns to petty theft, joins a gang, seeks alms or gives them. Each fact becomes one token. The town's facts are encoded once and cached; each question runs only the decision layer, in which one token per permitted answer attends to the question's facts and to the cached town. The bars are the shipped student's answer to this question.

Summing a triple's three embeddings into one token departs from the Triple World Model, which gives each of entity, attribute and value its own token and binds them with position embeddings. In Hearthvale each scope describes one entity, so the sum already binds a value to its attribute and scope, and the single-token form needs a third of the tokens. We nevertheless retained the original layout as one of the alternatives, alongside three models representing the traditions described in Section 2: a multilayer perceptron over the summed triple embeddings, with dropout tuned for a fair comparison; a factorization machine whose option scores

weight pairwise, and optionally third-order, interactions between triples, computed in linear time; and gradient-boosted trees, fitted separately for each kind on one-hot features.

5 Method

5.1 Data

Training data was gathered in two stages. The first provides breadth. We grew sixty-four towns automatically to their 150th year across seven kinds of terrain, each governed by a stand-in council that ruled on every proposal in one of ten political styles, from frugal to mercantile to pious. A recording layer captured every question exactly as the game client would pose it and answered it with the fallback rules so that the towns could proceed. Because these simulations do not step each resident individually, residents' immediate needs were static, and we added copies of each resident question with randomly drawn need levels so that the student would see the full range. These runs produced several million distinct questions spanning hamlets to late-era cities.

The second stage follows DAgger. Sixteen further towns were grown under the control of a first student, so that the data also covers the states a student's own decisions lead to, which differ from those reached under any other policy. In both stages the teacher answered a sample of the questions through the game's own request path, so that the prompts it saw were identical to those the game sends in play. Sampling took up to 15,000 questions of each kind, more for the frequent routine decision, and drew several questions from each snapshot, which let the teacher's server reuse its encoding of a shared snapshot. A third set, used only for evaluation, comes from four towns that the teacher itself governed for 150 years. In total the teacher labelled 460,701 questions.

5.2 Training

All neural students minimise the divergence from the teacher's distribution over the permitted options. Each question is weighted by the square root of how often the simulated towns asked it, so that common situations count for more without overwhelming rare ones. Training uses the AdamW optimiser with a one-cycle learning-rate schedule for twenty epochs, and keeps the epoch with the lowest error on held-out towns. The cheaper students can additionally learn from the set model: once trained, it labelled 1.2 million further questions from the training towns, and the factorization machine and perceptron were retrained on these labels together with the teacher's.

5.3 Evaluation protocol

Average error on held-out data is a necessary test but an insufficient one, so we evaluated the students in four increasingly demanding ways. The first measures error on towns excluded from training. Twenty per cent of towns are held out entirely, and because snapshots change every season, nearly every held-out question presents a snapshot never seen in training.

The second targets compositional generalization directly. For every pair of a question fact and a snapshot fact, we compared the teacher's average answer when both were present with an additive prediction from each fact alone, and chose four pairs whose joint effect departs most from additivity, together with two weakly interacting pairs as controls. Every training example containing both facts of a pair was removed, while examples containing either fact alone were kept, and we measured error on exactly the removed examples.

The third measures the student on questions from towns the teacher governed. These towns were grown headless, with residents' needs synthesised as in the first stage, so they are not play by people; but their states are those a town reaches under the teacher's policy, which is what a faithful student must handle, and they resemble the deployed game far

more closely than towns run by the fallback rules. We call them teacher-governed towns. Alongside the divergence, weighted both uniformly and by how often each question is actually asked, we compare the average probability the student and the teacher assign to each answer, and the ratio of the two for answers the teacher gives rarely.

The fourth compares complete towns over 150 years. Because the game samples its answers with a seeded random generator, the same town governed twice by the same model replays identically; to measure the natural variation of a town, we re-seeded its random stream immediately after creation, so that runs share a map and a starting point and differ only in their draws. Four towns were grown three times each under the teacher and eight times each under the student. For each outcome we report two quantities. The first is location: the mean difference between student and teacher runs of the same town, averaged over towns, with a bootstrap interval over runs and a permutation test that exchanges the labels of runs within each town. The second is spread: the pooled within-town standard deviation of the student's runs divided by that of the teacher's, which is about one when the student's towns vary as much as the teacher's. Both are needed. A single ratio of typical distances, student to teacher over teacher to teacher, rewards a student with no variation at all: one that always reproduced the teacher's median town would score about 0.71 on a normally distributed outcome and appear closer to the teacher than the teacher is to itself.

5.4 Deployment in the browser

Students ship as compact bundles in which each weight matrix is stored as eight-bit integers with one scale per row, a change that alters answers by less than one ten-thousandth of a nat. Every bundle carries a set of test questions with the answers its training framework gives, and a shipped model must reproduce them in JavaScript to within seven parts in ten million. The runtime caches each snapshot's encoding and shares it across every kind of question that uses the same snapshot. Because a month's

opening sweep can pose dozens of questions in a single simulation step, the runtime limits the student to a few milliseconds of work at a time and completes the remainder on subsequent steps, much as answers from the network would arrive.

5.5 Learning after release

A fixed student will eventually meet towns its training never covered, so the deployed system continues to learn from the teacher. A small auxiliary network, trained on held-out towns, predicts the student's own error on each question from the internal state that produced its answer, the answer's entropy and the share of unknown inputs; a single scale fitted on the held-out data makes its output read in nats. Questions whose predicted error lies in the top twentieth, questions containing values the student has never seen, and questions of kinds it does not answer are sent, deduplicated and without any player identifier, to the game's server. The server validates them, records a salted hash of each client so that distinct reports can be counted without identifying anyone, and obtains the teacher's answer as soon as the GPU is idle.

A job that wakes every hour turns these reports into the next version once at least fifty new labelled questions are waiting. A single report is enough for a question to be used. Labels come only from the teacher, so one player can at most direct the teacher's attention to unusual states, and the comparisons described below judge the result; requiring several players to report the same question would, in a game whose states rarely repeat exactly, discard nearly everything. What one player can contribute is bounded instead. A cycle takes at most 400 questions from any one client, in the order they arrived, and defers the rest to later cycles; questions whose categorical facts form more than two pairs never produced by any simulated town may make up at most a quarter of a cycle unless three clients reported them; and questions already trained on are skipped. The job then warm-starts the next version from the current one, appending new vocabulary entries initialised from the unknown token rather than

retraining from scratch, retrains the error estimate, and compares the candidate with the current version on error in teacher-governed towns, error on every frequently asked kind, the largest bias on any answer, rare-answer ratios, error on held-out new questions, browser parity, size, speed and a set of short simulated towns. The candidate is published only if it passes every comparison. A new town adopts the latest version, while a saved game keeps the version it began with, so that no town changes character partway through.

When the teacher is available, the browser also refers the questions it is least certain of directly to it. Each player may send at most one such question per second, always the least certain first, so the share referred falls as the game speeds up; the server enforces a further budget shared by all players and refuses the excess with a time to wait, in which case the student's own answer, already computed, is used.

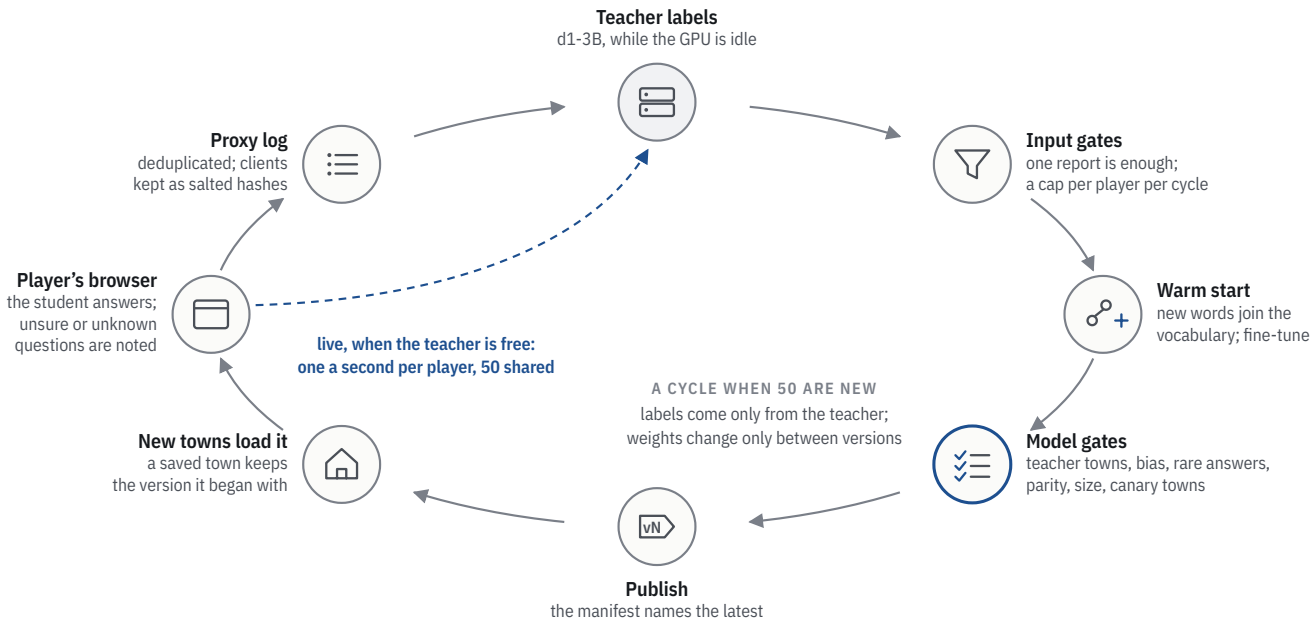


Figure 3. The continual-learning loop. Labels come only from the teacher, and one player's report is enough, within a per-player cap; weights change only between versions, and each saved game keeps the version it began with.

6 Results

6.1 Architectures compared

The architectures were first compared on the broad first-stage data, under identical conditions. The set model reproduces the teacher most closely, with a divergence of 0.025 nats on towns it never saw and agreement on the most likely answer in 86% of questions. Gradient-boosted trees come close in accuracy, but a model covering every kind requires about 12.7 megabytes of trees, some forty times the size of the set model. The factorization machine is somewhat less accurate yet exceptionally cheap, and learning from the set model's labels closes part of the gap. The multilayer perceptron, even after tuning, is the least accurate by a wide margin. Reducing the set model to a single layer in each stage costs almost nothing in accuracy and halves its running time, which makes it the natural candidate for deployment.

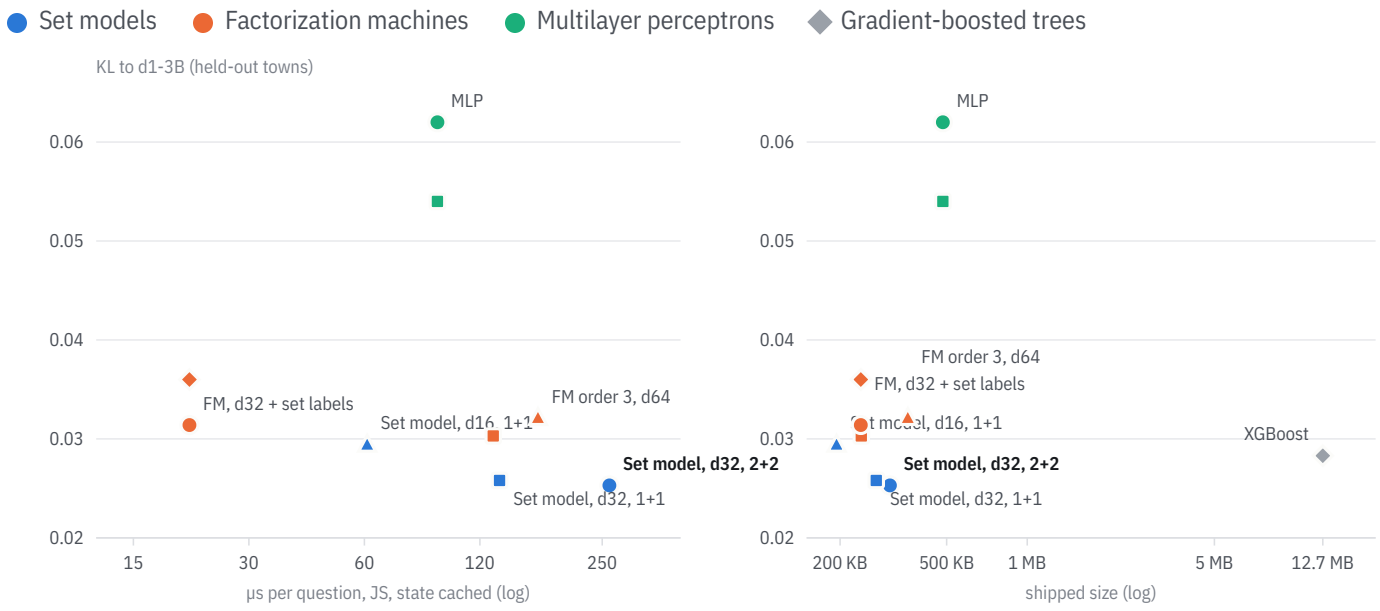


Figure 4. Divergence from the teacher on held-out towns against cost per question in the browser (left) and the size of the shipped model (right). Lower and further left is better. Gradient-boosted trees have no browser implementation and appear only on the right.

Table 2. Architectures compared on 30,667 questions from held-out towns of the first stage. The baseline, which answers every question with its kind's average distribution, scores 0.150.

Student	Divergence (nats)	Top answer agrees	Shipped size	Time per question
Set model, two layers per stage	0.0253	85.7%	307 KB	~260 µs
Set model, one layer per stage (shipped)	**0.0258**	**85.3%**	**273 KB**	**~130 µs**
Set model, small (d = 16)	0.0294	84.4%	194 KB	~60 µs
Gradient-boosted trees	0.0283	85.4%	~12.7 MB	not built
Factorization machine, with set-model labels	0.0314	84.0%	239 KB	~21 µs
Factorization machine	0.0360	82.6%	239 KB	~21 µs
Multilayer perceptron, best of four	0.0620	80.5%	484 KB	~93 µs

6.2 Compositional generalization

The test of withheld pairs separates the architectures sharply and bears out the reasoning of Section 4.1. On combinations of facts it never saw together, the set model's error is only 4% higher than on ordinary questions. The factorization machines lose between 13% and 15%, consistent with their design, and they perform best of all on the purest two-way interactions, such as craftsmen weighing a proposal that favours craft. The multilayer perceptron's error, by contrast, nearly quadruples, and on one pair, young people choosing a trade when farms offer no work and wellbeing is poor, its answers are scarcely better than the kind's average.

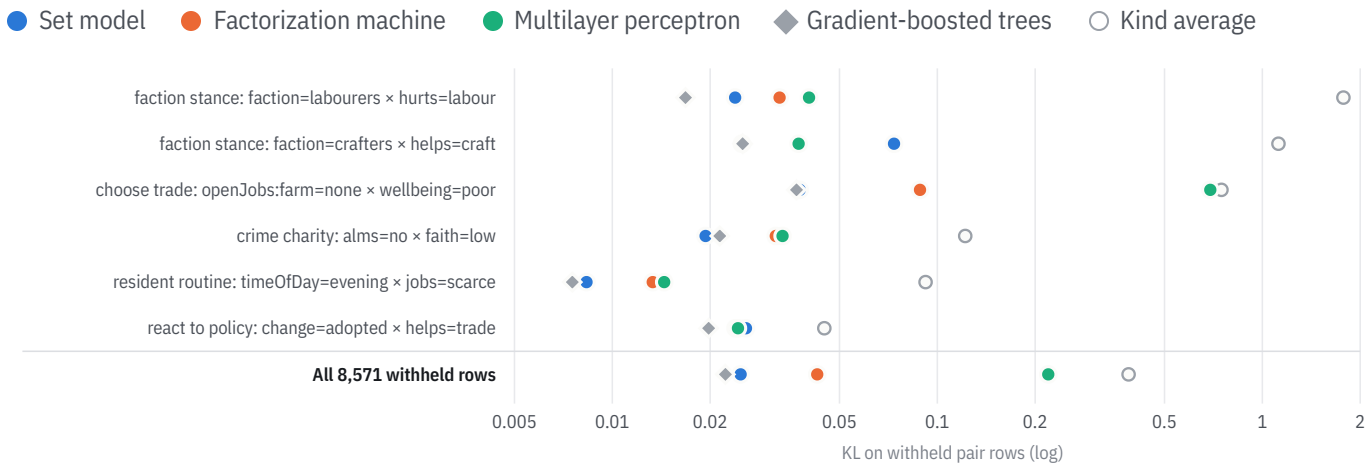


Figure 5. Divergence on questions whose pair of facts was withheld from training (logarithmic scale). Hollow markers show the kind-average baseline.

6.3 The final students on teacher-governed towns

The students intended for deployment were trained on both stages of data. Table 3 reports them on held-out towns from both stages and on the teacher-governed towns, which played no part in training. The deployed set model reaches 0.020 nats there, and 0.012 when each question is weighted by how often the game actually asks it, since the most frequent decisions, such as a resident's daily routine, are also among the easiest. Version 2, produced by the first cycle of the learning loop, differs from version 1 by 0.0003 nats on these towns (paired 90% interval 0.0002 to 0.0003, resampling snapshots): a consistent difference, but about 1%, and too small to matter in play. Its gain lies in the questions it was built for, reported in Section 6.6.

Table 3. The final students, trained on both stages of data. Teacher-governed towns: 40,119 questions from four headless towns the teacher governed, never used in training. Times are with the town's encoding cached and vary by about a tenth between runs on one machine.

Student	Held-out towns	Top answer agrees	Teacher-governed towns	Same, by traffic	Time per question
Set model, version 2 (current)	**0.0216**	**86.4%**	**0.0201**	**0.0119**	**~125 µs**
Set model, version 1	0.0217	86.4%	0.0204	0.0122	~125 µs
Set model, small (low-end tier)	0.0256	84.7%	0.0265	0.0155	~60 µs
Factorization machine, with set-model labels	0.0256	85.4%	0.0265	0.0156	~21 µs

Because the game samples from these distributions rather than taking the most likely answer, we also compared the average probability the student and the teacher assign to each answer. For the daily routine, which accounts for nearly three quarters of all questions, every answer lies within one percentage point of the teacher; across all frequently asked kinds no answer differs by more than about two points, and rare outcomes such as protest, emigration or a large loan are predicted at rates within a few percent of the teacher's. Training without the second, on-policy stage leaves the error on teacher-governed towns about a fifth higher, concentrated in household decisions, which supports gathering states from the student's own play.

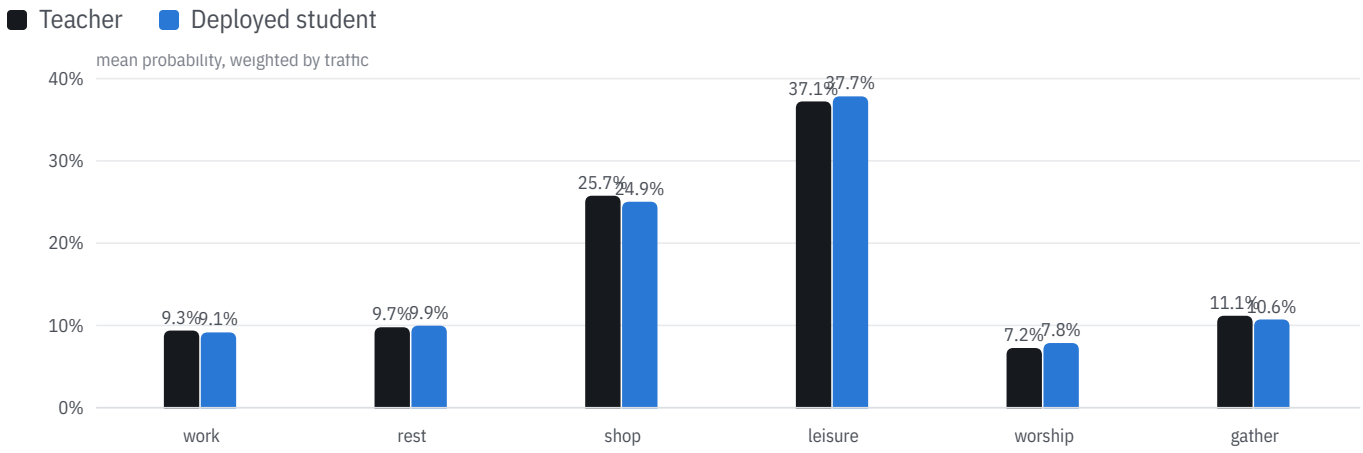


Figure 6. Average probability of each answer to a resident's daily routine on questions from teacher-governed towns, weighted by how often each question is asked: the teacher and the deployed student.

6.4 Behaviour of complete towns

The decisive test is whether towns behave alike. Grown three times each under the teacher, four towns showed how strongly chance shapes a town's history: one of them ended its 150th year with 1,098, 3,897 and 12,896 inhabitants in its three runs. Table A3 compares student and teacher runs of the same towns. In location, the student's towns cannot be told apart from the teacher's at this sample size: on population at every stage, the share of residents at work and wellbeing, the permutation test gives p-values between 0.08 and 0.78. The largest shift, a smaller population at the 150th year (−0.46 in log population, 90% interval −0.89 to −0.01), lies at the edge of significance and may reflect a slight residual tendency toward emigration. The fallback rules, by contrast, shift population by a factor of about four and the morning work share by 37 points, both clearly ($p \approx 0.03$ and 0.004), even though their population varies so much under the teacher that a ratio of distances barely registers it.

In spread, the student's towns vary less than the teacher's. The within-town standard deviation of log population is 0.41 to 0.75 of the teacher's, and that of the evening work share 0.25; the 90% intervals exclude one at the 100th year and for the evening work share. With three teacher runs per town the

teacher's own spread is poorly estimated, so the size of the gap is uncertain, but its direction matters for a game in which the same start should lead to different towns. The fair summary is that the student reproduces the average course of the teacher's towns, and that its towns may be more alike than the teacher's; more teacher runs, which the loop can grow while the GPU is idle, will settle by how much.

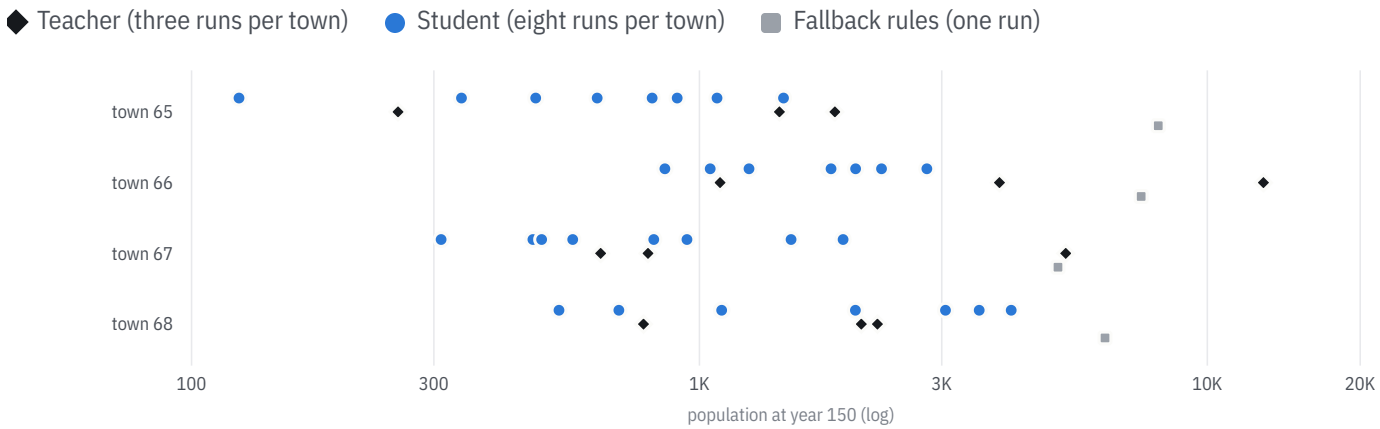


Figure 7. Population in the 150th year for every run of four towns unseen in training (logarithmic scale). Runs of a town share its map and starting point and differ only in their random draws.

6.5 Cost

The deployed student answers a question in about 125 microseconds on a single processor core, between 112 and 129 across runs on one machine, against five to nine milliseconds of GPU time per question for the teacher. The shipped bundle of version 2 is 363 kilobytes: 186 of model weights, 12 for the error estimate and 143 of vocabulary. The 273 kilobytes and 130 microseconds of Table 2 belong to the first-stage model, which has a smaller vocabulary and no error estimate. Grown end to end, a town governed by the student took between one and a half and five minutes to reach its 150th year, against some forty minutes for a town waiting on the teacher, and cost about as much as running with no model at all, because the simulation itself dominates. The heaviest single step observed, in a town of about a thousand

people, carried fifty-five questions and twenty-four milliseconds of student work at the opening of a month, which the per-step budget spreads over subsequent steps.

6.6 Confidence and the first learning cycle

The student's estimate of its own error proved useful. Its rank correlation with the actual error on teacher-governed towns is 0.53; marking the 5% of questions it judged most uncertain captured 42% of its worst errors, eight times what random selection would achieve, and its predictions match the observed error closely across the range. The first cycle of the learning loop ran on the questions reported during a single four-minute session of the game in a browser: 307 questions from one client, chiefly where to build and how factions stand, each labelled by the server within seconds of being reported. They are the only data from play in a browser so far, and they are not a sample of play: each was reported because the student was unsure of it or met an unknown value. Version 1's error on them is 0.080 nats (90% interval 0.065 to 0.097, resampling snapshots), four times its error on teacher-governed towns, which is what the selection should produce. Version 2 trained on four fifths of them. On the 68 it held out, its error is 0.086 against version 1's 0.095, a reduction of a tenth (paired interval for the difference −0.018 to −0.001). It passed every comparison against its predecessor and published itself.

7 Discussion

7.1 What the triple route contributed

The results bear out the properties that motivated the approach. The ordering of the architectures follows from the structure of the decisions: most of the teacher's answers depend on individual facts and on pairs of facts, such as a faction and the interests a proposal favours, or a trade and the jobs available. Attention and factorization both compute products of

features, whereas a multilayer perceptron over a sum of embeddings must approximate such products with successive nonlinear layers, which demands more data than the problem supplies. Factorization machines capture pairs efficiently but assign each pair a fixed strength, while attention can make an interaction depend on a third fact and disregard irrelevant ones. The resulting order, set model, then factorization machine, then perceptron, mirrors how much multiplicative structure each design builds in, and the withheld-pair test shows the same order with the largest margins.

The economy of the representation proved equally valuable in practice. A single set model of about 125,000 parameters answers every kind of question; the snapshot encoding it shares across questions keeps the common case cheap; and because new facts are new tokens, the learning loop extends the vocabulary and fine-tunes rather than rebuilding. These are the properties the Triple World Model was designed to demonstrate, and they transfer to a commercial setting with little modification.

7.2 The three-token layout

The Triple World Model's original layout, which writes each triple as three separate tokens, learned more slowly here and was set aside. In its original setting a state holds several entities sharing the same attributes, and relating values to entities across tokens is the problem attention solves. In Hearthvale each scope describes a single entity, so summing a triple's three embeddings already binds them, and the three-token form must instead learn that binding from position, at three times the cost per question. The representation transferred; the tokenization did not need to.

7.3 Evaluating a policy that acts repeatedly

The evaluation design deserves emphasis. Because a game samples from the student's distribution at every step for a century, a small and consistent bias on one answer can alter a town more than larger random errors, and

average divergence alone cannot reveal it. For this reason the evaluation compares the probability of each answer as well as the overall divergence, measures the student on the states its own decisions produce, and compares complete towns with the teacher's in both average and spread. The last point is easy to get wrong: a single ratio of distances between runs cannot distinguish a faithful student from one whose towns are merely less varied, and in this work it hid exactly that. We regard these measures as necessary for distilling any policy that acts repeatedly within a simulation.

7.4 The teacher's character

The experiments revealed something about the teacher itself. Its answers keep towns markedly smaller than the fallback rules do, chiefly because it sends few residents to work. The students copy this faithfully, which is the intended behaviour, but it is a property of the teacher that the game's designers may wish to examine, and one that a distilled student now makes cheap to study.

7.5 Limitations

The comparison of complete towns rests on four towns and three teacher runs of each, enough to separate regimes but not to detect small shifts such as the possible difference at the 150th year, nor to estimate the teacher's spread precisely. Simulated towns do not step every individual resident, so residents' needs were synthesised, and the teacher-governed towns are headless; the only questions from play in a browser come from one short session. Timings come from a single fast machine, and a mid-range phone will be several times slower, although the per-step budget accommodates this. The withheld-pair test covers six pairs chosen for strong interactions, and the error estimate misses more than half of the worst questions when it marks only a twentieth of them.

8 Conclusion

A small model of structured triples can take the place of a large language-model decision policy for the great majority of a game's decisions. It matches the teacher closely, generalizes to unfamiliar combinations of circumstances, reproduces the average course of the teacher's towns, and costs a small fraction of the GPU time it replaces. Three elements made this possible: a representation, inherited from the Triple World Model, that exposes the structure of each question; an evaluation grounded in the states the deployed policy reaches and in complete towns rather than in average error alone; and a loop through which the student continues to learn from its teacher after release. The same approach should extend to other games and simulations that rely on language models to animate their agents, wherever the questions put to those models are structured and the cost of answering them scales with the audience.

References

1. G. Hinton, O. Vinyals and J. Dean. Distilling the knowledge in a neural network. *NIPS Deep Learning Workshop*, 2015.
2. A. A. Rusu et al. Policy distillation. *International Conference on Learning Representations*, 2016.
3. T. Chen and C. Guestrin. XGBoost: A scalable tree boosting system. *KDD*, 2016.
4. Y. Gorishniy, I. Rubachev, V. Khrulkov and A. Babenko. Revisiting deep learning models for tabular data. *NeurIPS*, 2021.
5. A. Beutel et al. Latent cross: Making use of context in recurrent recommender systems. *WSDM*, 2018.
6. S. Rendle. Factorization machines. *IEEE International Conference on Data Mining*, 2010.
7. M. Zaheer et al. Deep sets. *NeurIPS*, 2017; J. Lee et al. Set Transformer. *ICML*, 2019.
8. S. Ross, G. Gordon and D. Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. *AISTATS*, 2011.

9. Y. Geifman and R. El-Yaniv. Selective classification for deep neural networks. *NeurIPS*, 2017; D. Madras, T. Pitassi and R. Zemel. Predict responsibly: Improving fairness and accuracy by learning to defer. *NeurIPS*, 2018.
10. Cloudflare. Clef-flash model card. huggingface.co/Cloudflare/clef-flash, 2026.
11. J. Palmer. Kev: open decision models. github.com/jaredpalmer/kev, 2026.
12. Convai Innovations. Laya: a non-autoregressive System 1 decision engine. github.com/NandhaKishorM/laya, 2026.

Appendix A. Detailed results

Table A1. Divergence on held-out towns and on withheld pairs, from a training run with the six pairs removed.

Student	Held-out towns	Withheld pairs	Ratio
Set model	**0.0238**	**0.0248**	**1.04×**
Set model, small	0.0288	0.0332	1.15×
Factorization machine, third order	0.0324	0.0366	1.13×
Factorization machine	0.0372	0.0427	1.15×
Multilayer perceptron	0.0583	0.2194	3.76×
Gradient-boosted trees (five kinds)	—	0.0223	—

Table A2. Divergence on questions from teacher-governed towns, for students trained on the first stage of data only and on both stages, overall and weighted by how often each question is asked.

Student	Held-out towns	First stage: teacher-governed	First stage: by traffic	Both stages: teacher-governed	Both stages: by traffic
Set model, two layers per stage	0.0253	0.0232	0.0145	—	—
Set model, one layer per stage (shipped)	**0.0258**	**0.0242**	**0.0164**	**0.0204**	**0.0122**
Set model, small	0.0294	0.0292	0.0184	0.0265	0.0155
Factorization machine	0.0314	0.0321	0.0185	0.0265	0.0156
Factorization machine, third order	0.0303	0.0311	0.0193	—	—
Multilayer perceptron	0.0620	0.0613	0.0336	—	—

Table A3. Complete towns: the student's and the fallback rules' runs against the teacher's runs of the same four towns. Shift is the mean difference from the teacher, with a 90% bootstrap interval and a permutation p-value; spread is the student's within-town standard deviation over the teacher's, with a 90% interval. The fallback rules ran once per town, so their spread is undefined. Population is compared on a logarithmic scale.

Outcome	Student shift	p	Student spread	Fallback shift	p
Population, year 50	+0.22 [−0.07, +0.53]	0.25	0.75 [0.59, 1.60]	+1.01 [+0.76, +1.28]	0.024
Population, year 100	+0.07 [−0.39, +0.54]	0.78	0.41 [0.33, 0.93]	+1.44 [+0.97, +1.92]	0.030
Population, year 150	−0.46 [−0.89, −0.01]	0.08	0.63 [0.51, 1.34]	+1.40 [+0.98, +1.79]	0.034
Morning work share (points)	−0.85 [−1.96, +0.15]	0.16	0.65 [0.49, 2.19]	+36.93 [+35.93, +37.85]	0.004
Evening work share (points)	−0.20 [−0.53, +0.12]	0.20	0.25 [0.19, 0.80]	+5.61 [+5.26, +5.93]	0.004
Wellbeing, year 150	−2.05 [−7.10, +3.00]	0.61	1.10 [0.70, 4.15]	+7.42 [+3.75, +11.42]	0.195

Table A4. Questions marked by the error estimate, on teacher-governed towns.

Questions marked	Share of all error	Share of the worst 5%	Random marking
top 1%	7.4%	13.0%	1%
top 5%	**23.8%**	**42.0%**	**5%**
top 10%	38.5%	66.1%	10%

Table A5. Training and deployment settings.

Setting	Value
Set model (deployed)	width 32, one encoder layer, one decision layer, two attention heads, feed-forward width 64; about 125,000 parameters
Optimiser	AdamW, one-cycle schedule, peak learning rate 0.003, 20 epochs, batch 1,024; best epoch on held-out towns
Question weight	square root of how often simulated towns asked the question
Held-out towns	20% of towns, chosen by a hash of the town's seed
Shipped weights	8-bit integers with one scale per row; error estimate in 32-bit floats
Browser budget	4 ms of student work per turn of the event loop
Live referrals	at most one per second per player; a shared server budget, 50 per second by default
Loop intake	one report suffices; at most 400 questions per client per cycle, the rest deferred; unusual states at most a quarter of a cycle unless three clients report them; questions already trained on skipped; a cycle when 50 new labelled questions wait, checked hourly
Loop gates	no more than 3% worse on teacher-governed towns overall or by traffic; no frequent kind more than 20% worse; largest answer bias no more than 5 points; rare-answer ratios between 0.67 and 1.5

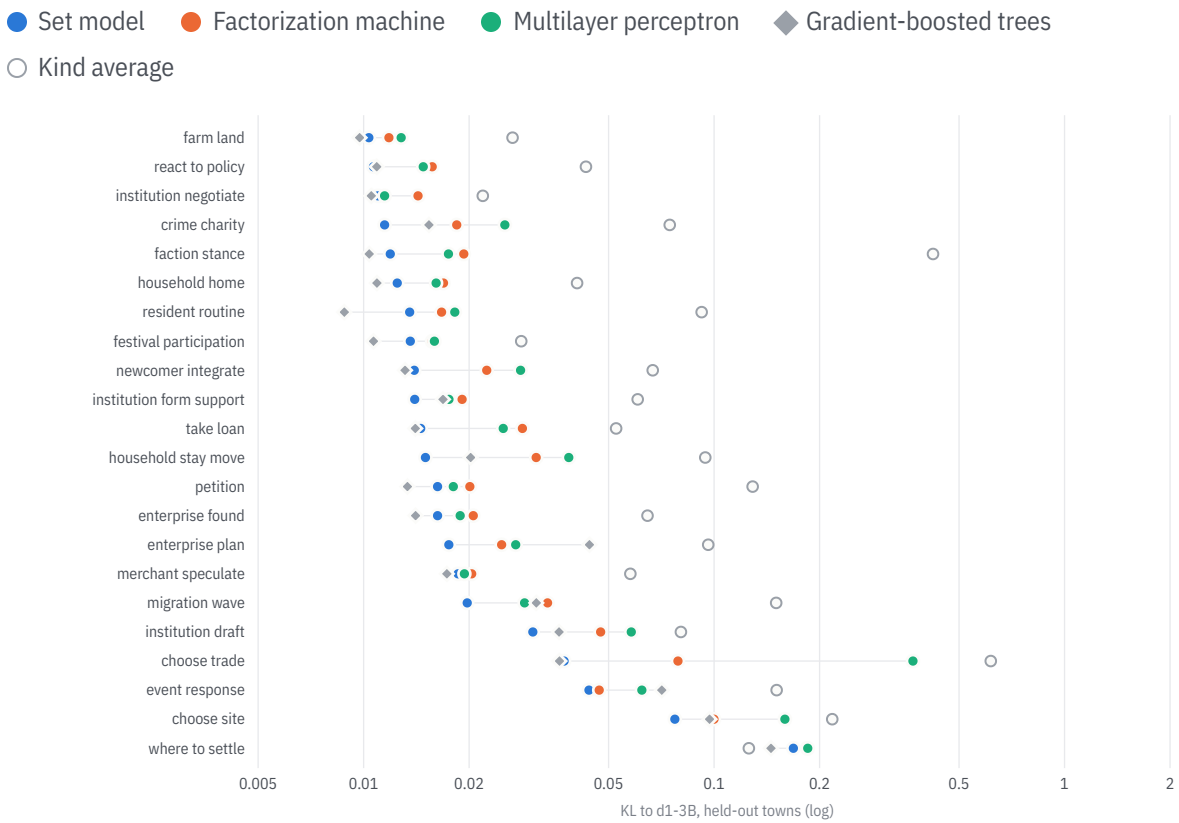


Figure A1. Divergence by kind of decision on held-out towns of the first stage (logarithmic scale). The hardest kinds are those whose options are free text that changes from question to question.

Code: Hearthvale `scripts/distill-*.js`, `src/decision-features.js`, `src/decision-student.js`, `src/decision-local.js`; the training code lives with the Triple World Model in `src/twm/decision/` and `scripts/train_decision_student.py`. Every shipped student matches its PyTorch original within 7×10^{-7} in the browser.